openAG Cyber-Physical Extensions (Normative)

Status: Normative draft — design-only. Additive extensions to the openAG standard that let the graph capture multi-disciplinary cyber-physical systems (controls + mechanical + electrical + software) and serve as a digital-twin backbone. openAG remains zero-code; AMS frozen until parity. These extensions are documentation + schema only.

Date: 2026-10-12. Conformance language: MUST / SHOULD / MAY per RFC 2119. Companion to `docs/openAG/standard.md` (the core contract).

Forcing function. This extension set was derived by hand-authoring the Heller Cell (5 CNC machines + auto tool-changers, 2 robots, part/tool stores, coolant, multiple PLCs, robot code, comms, SCADA/servers/DBs) as the first openAG reference model (`docs/plans/2026-10-12-heller-cell-openAG-design.md`). Each extension here closes a gap the cell exposed in the base standard.

Backward-compatibility. All additions are additive. A conformant base (software-only) openAG graph remains conformant; domain profiles, the A8 axis, and the twin vocabulary are opt-in. A graph that uses none of them behaves exactly as under `standard.md`.

---

1. Why the base standard is insufficient

The base standard (`standard.md`) defines software-flavored node kinds and seven axes with no geometry. A physical manufacturing cell breaks three assumptions on contact:

base assumption	cyber-physical reality	extension
Node kinds are software (Component/Interface/…).	A cell has sensors, actuators, PLCs, drives, robots, machine-tools, pumps.	§2 Domain profiles
Decomposition is non-spatial (7 axes).	A twin needs pose, mounting, adjacency, coordinate frames.	§3 A8 Spatial axis
Promises are prose/structural claims.	A twin needs geometry refs, kinematics, live-data bindings.	§4 Twin vocabulary
Four fact-source oracles.	Geometry is grounded by CAD/metrology, not code-scan or runtime.	§5 Oracle set

---

2. Domain profiles

A domain profile extends the base node vocabulary with discipline- specific subkinds, attributes, and validation rules, mirroring the AMS profile mechanism (`controls` / `mechanical` / `electrical`). A graph declares active profiles in `meta.domain_profiles` (a list — net-new vs AMS's single `domain_profile`, because a cyber-physical system is inherently multi-disciplinary).

```yaml
meta:
  domain_profiles: [controls, mechanical, electrical, software]
```

Component.subkind (normative). Every Component MAY carry a `subkind` drawn from the union of active profiles' subkind vocabularies. A Component MAY also carry a `discipline` tag (`controls | mechanical | electrical | software`) for cross-discipline queries.

2.1 **`controls`** profile

facet	values
subkinds	`sensor`, `actuator`, `controller`, `hmi`, `plc-program`, `drive`, `relay`, `safety-controller`
interface subkinds	`fieldbus`, `signal`, `analog-io`, `digital-io`, `modbus`, `profinet`, `ethercat`, `opc-ua`
behavior patterns	`feedback-loop`, `interlock`, `motion-profile`, `fault-tolerance`, `cascade-control`
constraint types	`safety`, `timing`, `control-accuracy`, `availability`
attributes	`signal_type` (analog|digital|fieldbus|discrete), `sampling_rate_hz`, `safety_integrity_level` (0–4), `io_address`
rules	`sensor` and `actuator` MUST declare `signal_type`.

2.2 **`mechanical`** profile

facet	values
subkinds	`part`, `assembly`, `subassembly`, `fastener`, `fixture`, `bearing`, `gear`, `shaft`, `housing`, `frame`
interface subkinds	`mechanical`, `bolt-pattern`, `press-fit`, `weld`, `adhesive`, `threaded`, `keyed`, `splined`
behavior patterns	`motion-profile`, `kinematic-chain`, `load-transfer`, `thermal-cycle`, `fatigue-cycle`
constraint types	`material`, `load`, `tolerance`, `thermal`, `fatigue`, `weight`, `clearance`
attributes	`material`, `mass_kg`, `dimensions_mm` {length,width,height}, `tolerance_class` (IT6–IT12)
rules	`part` MUST declare `material`.

2.3 **`electrical`** profile

facet	values
subkinds	`pcb`, `connector`, `power-supply`, `cable`, `terminal`, `breaker`, `transformer`, `vfd`
interface subkinds	`power`, `signal-cable`, `connector-mating`, `bus-bar`, `harness`
constraint types	`voltage`, `current`, `power`, `insulation`, `emc`
attributes	`voltage_v`, `current_a`, `power_w`, `phase` (1|3), `protection_class` (IPxx)
rules	`power-supply` MUST declare `voltage_v`.

2.4 Cell-specific subkinds (manufacturing)

The Heller cell needs a few manufacturing subkinds not in the base AMS profiles. These are added to the `controls`/`mechanical` profiles:

machine-tool, robot, tool-changer, pump, conveyor, gripper, spindle, store (part/tool magazine).

---

3. A8 — the Spatial / Geometric axis

The base standard's seven axes (A1–A7) carry no geometry. openAG adds an eighth axis:

> A8 — Spatial. Governs physical placement and connection: pose, mounting, adjacency, mating, and routing in physical space.

A8 relationship types (normative):

type	meaning	example
mounted-on	source is physically fixed to target.	spindle mounted-on machine frame.
adjacent-to	source neighbours target in the layout.	CNC-1 adjacent-to CNC-2.
mates-with	source mechanically couples to target.	gripper mates-with part fixture.
connected-at	source connects to target at a port/point.	coolant line connected-at spindle inlet.
routed-through	source passes through target's space/raceway.	power harness routed-through cable tray.

A8 is orthogonal to A1: A1 contains is logical/abstraction containment (a partition); A8 is physical placement. A pump may be A1- contained by SUB-COOLANT yet A8 mounted-on a frame that belongs to a different physical assembly. Keeping A1 and A8 distinct is a core design assertion of this extension (see the design doc's finding #6).

Coordinate frames. A8 introduces a CoordinateFrame concept: a named reference frame (cell-origin, robot1-base, cnc3-table, …) with an optional parent frame and a rigid transform. Component pose (§4.1) is expressed relative to a named frame, forming a frame tree rooted at cell-origin.

---

4. Twin promise vocabulary

A digital twin hangs off the graph when components carry geometry, kinematic, and live-binding promises. These are ordinary promises (attribute / content / status / axis / valence) whose content follows a typed shape. Heavy payloads (meshes, point clouds) are never inlined — only referenced by URI, consistent with the CON-N2 NN-compilable export.

4.1 Geometry promises (**axis: A8**)

attribute	content shape	grounds
geometry_ref	URI to CAD/mesh (STEP/glTF/USD/STL).	the component's shape.
pose	{frame, translation:[x,y,z], rotation:[quat]}.	its placement in a frame.

attribute	content shape	grounds
`bounding_box`	`{frame, min:[x,y,z], max:[x,y,z]}`.	coarse extent (for culling/collision).
`coordinate_frame`	a `CoordinateFrame` declaration.	the frame itself.

4.2 Kinematic promises (**`axis: A8`** + **`A5`**)

attribute	content shape	grounds
`kinematic_chain`	ordered joints (URDF/DH-style): `[{name, type: revolute|prismatic, axis, limits, parent_frame, child_frame}]`.	articulated motion.
`motion_profile`	`{behavior_ref, accel, vel, jerk, waypoints?}`.	a programmed move.
`reach_envelope`	URI or parametric volume.	robot/axis workspace.

Kinematics are carried by Behavior nodes (`motion-profile` / `kinematic-chain` patterns from §2) linked to the moving Component along A8.

4.3 Live-binding promises (dynamic, **`axis: A7`**)

Every live-stateful component carries a dynamic promise (standard §5.2) whose `content` includes a binding descriptor:

```yaml
attribute: live_state
status: open            # verified iff runtime oracle confirms a live value
axis: A7
valence: "every actuated/stateful component MUST expose a live_state binding"
content:
  binding:
    protocol: opc-ua            # opc-ua | plc-tag | historian | mqtt
    node_id: "ns=3;s=CNC3.Spindle.RPM"   # or tag / historian key
    unit: rpm
    sample_hz: 10
```

A `verified` live-binding promise means the runtime oracle (§5) has confirmed a live value against the binding. Until then it is `open` (or `typed-gap` if the binding itself is unknown). This is how the twin becomes live without the model holding any runtime payload.

---

5. Oracle set for cyber-physical grounding

The base standard's four oracles (code-scan, human-dialogue, model-self, runtime) extend to physical systems, with one candidate 5th oracle:

oracle	actor	grounds	mode
code-scan	Target (PLC/robot programs)	control logic, tags, interlocks.	symbolic

oracle	actor	grounds	mode
human-dialogue	Modeler / Engineer	intent, layout facts, acceptance.	neural / human
model-self	the graph	derived/rolled-up facts.	symbolic
runtime	SCADA / historian / OPC-UA	dynamic state, live-binding promises.	executable
metrology / CAD (candidate)	CAD system / laser scanner / survey	geometry, pose, dimensions (A8).	static-evidence

Open question (finding-worthy). Is the 5th oracle genuinely distinct, or does geometry ground as ordinary static evidence produced by the code-scan oracle generalized to "artifact-scan"? The Heller cell authoring will record which framing holds. Until resolved, geometry promises ground via static evidence with `provenance: cad` / `provenance: metrology`.

---

6. Conformance (delta over standard §10)

A graph using these extensions additionally conforms iff:

9. Active domain profiles are declared in `meta.domain_profiles`; every Component `subkind` is drawn from an active profile's vocabulary; profile validation rules (§2) hold.
10. Every relationship and promise carries a valid `axis` including A8; A1 (logical containment) and A8 (physical placement) are kept distinct.
11. Geometry/kinematic/live promises follow the typed shapes in §4; heavy payloads are referenced by URI, never inlined; the graph still round-trips losslessly through `CON–N2`.
12. Live-stateful components expose a `live_state` dynamic promise with a binding descriptor (§4.3), grounded only by the runtime oracle.

---

7. Status and non-goals

- Design-only. No code; no schema file emitted yet. This is the contract a future implementation (and the hand-authored Heller model) satisfies.
- Projection is separate. These extensions let the model capture a twin's information; rendering, simulation, and live wiring are downstream and out of scope.
- Findings feed back. Every ambiguity flagged here (A8-vs-promise for geometry, the 5th oracle, the two-promise burden at scale) is logged in the model repo's `FINDINGS.md` and will refine this doc.